DSA4213 Assignment 1: Language Models, Embeddings, and Representations

[Zhang Yue Qian] [A0351049H]
Semester 1, AY2026/2027
github link:https://github.com/zzemm/DSA4213-Language-Models.git

1 Question and Setup

My question is: on one small corpus, how much do the model family and the input embeddings change next-token prediction, and does a better language model also give better features for sentiment classification? I train four language models on WikiText-2 (Part I), change the input embeddings of the LSTM (Part II), and use the three LSTMs as frozen feature extractors on IMDb reviews (Part III).

Data and preprocessing. Parts I and II use WikiText-2 raw (`wikitext-2-raw-v1` on Hugging Face) [1] with its official train, validation and test splits. I lowercase the text and split it with the regular expression `\w+|[^\w\s]`, so words and single punctuation marks become tokens. Empty lines are dropped and every other line ends with `<eos>`. This gives 2,145,904 training, 224,072 validation and 252,582 test tokens. The vocabulary is the 9,997 most frequent training tokens plus `<pad>`, `<unk>` and `<eos>` (10,000 in total), and other tokens become `<unk>`. The OOV rate is 7.6% on train, 9.2% on validation and 10.1% on test. The raw text writes hyphens and number separators as `@-@` and `@,@`, so each becomes three tokens. I kept them, and they show up in the generated text.

For the neural models each line is cut into non-overlapping pieces of at most 64 target tokens (48,136 train, 4,978 validation and 5,710 test segments). A model only sees earlier tokens of the current segment, and the hidden state starts fresh for each segment. Batches are padded with `<pad>`, and padded positions are left out of every loss.

Common evaluation. To compare all four models on exactly the same predictions, I score every test target that has at least two earlier tokens in its segment. This is 243,981 tokens for every model. I report the average next-token cross-entropy and the perplexity PPL = exp(loss). The trigram counts are built from the same training segments, so no model uses context across segment boundaries.

Hardware and timing. All runs used one Apple Silicon Mac (5, 16GB) with the PyTorch MPS backend (PyTorch 2.14.0, Python 3.11). Training time is wall-clock time (`time.perf_counter`) for all 8 epochs, including the validation pass after each epoch. For inference I use one common workload: score the first 256 test segments in batches of 32, three times, after one warm-up batch, with a device synchronize before the clock is read (the trigram runs on the CPU in plain Python). For the neural models I also timed a fixed workload: one random 32×64 batch, 30 forward passes.

2 Part I: Language Model Comparison

Models. *Trigram:* add-k smoothing, $P(w_i \mid w_{i-2}, w_{i-1}) = \frac{c(w_{i-2},w_{i-1},w_i)+k}{c(w_{i-2},w_{i-1})+kV}$ with $V = 10{,}000$. I chose k from $\{0.001, 0.01, 0.1, 0.5, 1\}$ on the validation set. Validation PPL grows with k (701 at $k = 0.001$, 2,393 at $k = 1$), so I use $k = 0.001$. *RNN and LSTM [2]:* two layers (tanh RNN or LSTM), embedding size 100, hidden size 256, dropout 0.2 between layers, and a linear output layer over the vocabulary. *Transformer [3]:* embeddings of size 100 projected to 256, learned position embeddings, two pre-norm layers with 4 heads and feed-forward size 512, dropout 0.2, a causal mask plus a padding mask, and a final layer norm. All neural models start from random, trainable embeddings. They are trained with Adam (learning rate 10^{-3}), batch size 32 and gradient clipping at 1.0 for 8 epochs. I keep the epoch with the lowest validation loss. The seed is 42 everywhere.

A note on the RNN row. The notebook kernel lost its in-memory state during the session, and the models were reloaded from their best checkpoints. Before that, a restarted training run was stopped in its third RNN epoch, after it had overwritten the RNN checkpoint with its epoch-2 weights. So

Table 1: Part I results. Loss and PPL use the common evaluation (243,981 test tokens). The last column scores all targets, including the first one of each segment, and was measured on the 8-epoch models before the kernel state was lost. Throughput is in tokens/s. For the trigram, "parameters" means stored count entries.

Model	Val loss	Test loss	Test PPL	Params (M)	Train time (s)	Thr. common workload	Thr. fixed batch	PPL all targets
Trigram	6.553	6.460	639.2	1.60	1.8	3,457,240	–	–
RNN[†]	4.855	4.784	119.6	3.79	626.3	107,498	313,459	99.5
LSTM	4.502	4.442	**84.9**	4.46	714.3	175,900	299,178	**85.6**
Transformer	4.517	4.450	85.6	4.78	669.5	161,662	247,547	86.1

[†] In the common-evaluation columns this row is the epoch-2 RNN checkpoint, not the 8-epoch model (see the note below). Its training time is for the full 8-epoch run.

the RNN in the common evaluation is a 2-epoch model. Its validation loss there (4.855) matches the epoch-2 value of the stopped run (4.854), while the 8-epoch RNN ends near 4.67 (Figure 1). The LSTM and Transformer checkpoints were not affected. The reloaded LSTM gives the same test PPL (85.606) as before.

Prediction quality. *Observed:* all neural models are far better than the trigram (85 to 120 against 639). The LSTM (84.9) and the Transformer (85.6) are almost tied. The 8-epoch RNN is clearly behind them (99.5 against 85.6 and 86.1 on all targets). The trigram is very sparse. With counts over the whole token stream it reaches PPL 22 on the training text but 675 on validation. There are 1.19 million distinct trigrams in 2.15 million training tokens, so most test contexts are rare or unseen, and add-k then spreads the probability almost evenly. *Possible explanations:* the neural models share statistics between similar words through the embeddings, and they can use up to 63 earlier tokens instead of 2. My setup does not separate these two causes. The RNN gap may come from its simple recurrence, which has trouble carrying information over many steps. Model size also differs. Most parameters sit in the embedding table (1.00M) and the output layer (2.57M), which are the same in all three models. The part that differs is small: 0.22M for the RNN, 0.89M for the LSTM and 1.21M for the Transformer. So the RNN also has the smallest core, and I cannot say how much of its gap is due to the architecture alone.

Training and inference cost. The trigram trains in 1.8 s and scores 3.46M tokens/s on the CPU, 20 to 32 times the speed of the neural models on the common workload. The neural models need 626 to 714 s for 8 epochs. On the common workload the LSTM (176k tokens/s) is a little faster than the Transformer (162k). The RNN is the slowest there (107k) but the fastest on the fixed batch (313k against 299k and 248k). The common workload rebuilds padded batches on the CPU, and the RNN was timed first, right after the trigram. I think its common-workload number includes warm-up and batching overhead (not tested), so for the RNN I trust the fixed-batch order more.

Learning behaviour (Figure 1). The Transformer's training loss falls fastest and ends lowest (about 4.04). Its validation loss improves very little after epoch 6 and is slightly higher at epoch 8 than at epoch 7. Its gap between training and validation loss is the largest (about 0.5), so it is starting to overfit. The LSTM starts slowest (training loss 5.88 after epoch 1), but its validation loss is still falling at epoch 8 (about 4.51). It has not converged and might pull ahead with more epochs. The RNN's validation loss flattens near 4.67 after epoch 5. Early validation losses are below the training losses because dropout is active in training and the training loss is averaged over the epoch.

Generated text (Table 2). All models sample at temperature 1.0 over the full vocabulary except `<pad>` and `<unk>`, for at most 30 new tokens, with a new CPU random generator seeded with 42 for each prompt and model.

- *Trigram.* After the first few words it outputs a list of unrelated words. From the fourth generated word on, the three trigram continuations are the same 27 words. Once the two-word context was never seen in training, add-k gives the same flat distribution, and the same seed then draws the same words. Greedy decoding (tried in an earlier cell) gets stuck in a loop: "of the `<unk>` of the

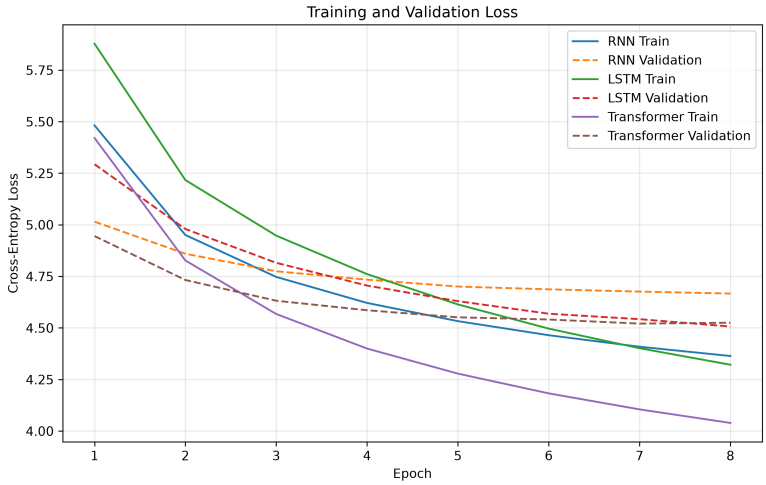


Figure 1: Part I training (solid) and validation (dashed) loss per epoch for the 8-epoch runs.

Table 2: Matched generated text (temperature 1.0, at most 30 new tokens, same seed). Continuations are cut with "…". All samples are in Appendix B. †Epoch-2 RNN checkpoint.

Prompt	Model	Continuation
the history of	Trigram	the earliest giant prices irony hergé marched principle part thailand quantity georgia …
	RNN†	the earliest government . the south of 250 @ , @ 500 people were dug down about 30 horses …
	LSTM	the roman government . the south end is part of the cathedral , who had created about 30 horses …
	Transformer	the fulbe government . the south vietnamese court part of the united states virgin center was listed in land …
in the world	Trigram	described viaduct delaware prices irony hergé marched principle part thailand …
	Transformer	war i , which was accused of the part of the people , written by local authors and education . <eos>

<unk> …".

- *Neural models.* None of them loops. Local word order is often fine, mostly for the LSTM and the Transformer ("the roman government . the south end is part of the cathedral"). The Transformer is the only model that ends a sentence and stops ("in the world war i , … education ."). It also produces correct name pairs such as "south vietnamese" and "north korean". The RNN copies WikiText formatting such as "250 @ , @ 500" and "(1915 cm)". No model stays on one topic for more than about one clause.
- *Shared seed.* "delaware prices" appears at the same positions for the trigram, the RNN and the LSTM. Matched seeds make side-by-side reading fair, but shared rare words reflect the random numbers more than the models.

Tradeoffs. On this corpus and budget the LSTM and the Transformer reach the same quality. The LSTM has fewer parameters and higher throughput on the common workload. The Transformer trained a little faster and fits the training data sooner, but it also overfits sooner. The trigram costs almost nothing to train and is 20 to 32 times faster at inference, but its PPL is 7.5 times higher and its text has no local fluency.

Table 3: Part II results (LSTM, embedding dimension 100). Validation losses for Random were read from the Part I plot (marked ≈).

Embedding	Trainable params	Test loss	Test PPL	Val loss epoch 1	Val loss epoch 8	LM train time (s)	Embedding cost
Random (trainable)	4,462,928	4.450	85.61	≈5.29	≈4.51	714.3	–
CBOW (frozen)	3,462,928	4.451	85.75	5.103	4.514	532.1	112.1 s training
GloVe (frozen)	3,462,928	4.404	**81.78**	5.319	**4.461**	622.2	128 MB download

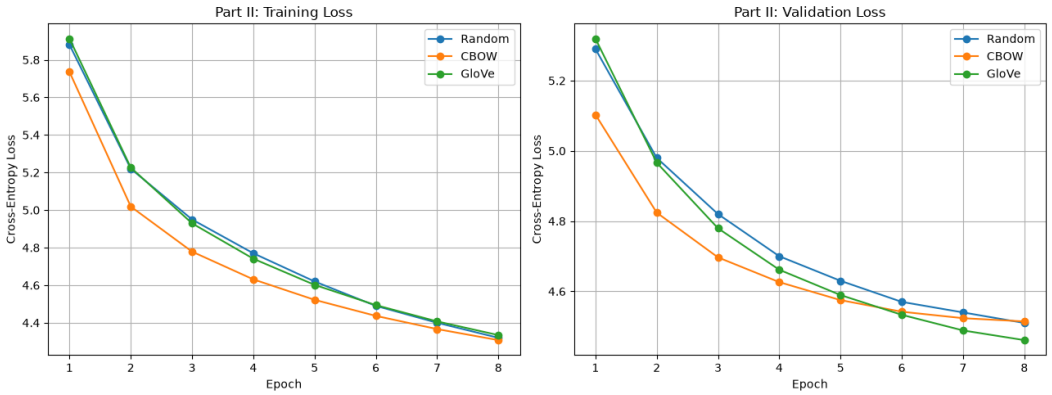


Figure 2: Part II training and validation loss (Random curve re-entered from the Part I plot).

3 Part II: Embedding Variants and Ablation

I use the LSTM from Part I and compare three input-embedding settings. All use dimension 100, which matches the public vectors. The architecture, data, optimizer, 8 epochs and best-validation checkpoint are the same in all three.

(a) *Random:* the Part I LSTM, reused, with embeddings trained together with the model.

(b) *CBOW (my own) [4]:* trained on the WikiText-2 training tokens with a window of 2 on each side. The model averages the four context vectors and applies a full softmax over the 10,000 words. I train it with Adam (10^{-3}), batch 512, for 5 epochs. I skip pairs that cross an `<eos>` or contain `<unk>`, which leaves 1,386,600 pairs. The loss fell from 6.25 to 4.86 (Figure 4). The `<unk>` and `<eos>` rows never get a gradient and keep their random start values, and `<pad>` is zero.

(c) *GloVe (public) [5]:* `glove-wiki-gigaword-100` from `gensim-data` [6], trained on 6B tokens of Wikipedia and Gigaword. It covers 9,919 of my 9,997 normal words (99.2%). The 78 missing words are mostly symbols such as °, × and →, plus rare names. These words, `<unk>` and `<eos>` get random $\mathcal{N}(0, 0.02^2)$ vectors, and `<pad>` is zero.

In (b) and (c) the whole embedding table is frozen, which removes 1,000,000 trainable parameters. All other weights are trained from scratch. Test PPL in this part scores all targets, like the last column of Table 1.

A note on the Random curve. The loss history of the Part I LSTM was lost with the kernel state, so I re-entered it from the saved Part I plot to two decimals. The Random curve in Figure 2 and its stability number are approximate. Its test PPL is exact, because it comes from the reloaded checkpoint.

Results. *Prediction quality.* GloVe gives the best test PPL (81.78), 4.5% below Random (85.61). CBOW is equal to Random (85.75). *Convergence.* CBOW starts best, with a validation loss of 5.10 after epoch 1 against about 5.29 for Random and 5.32 for GloVe. Its lead shrinks, and in the last epoch it improves by only 0.009. GloVe starts worst but keeps improving (0.028 in the last epoch). It passes CBOW at epoch 6 and ends lowest (4.461). GloVe has the highest training loss at epoch 8 (4.334, against 4.308 for CBOW) but the lowest validation loss. Its train-to-validation gap is the smallest: 0.13, against 0.21 for CBOW and about 0.19 for Random. *Stability.* All three validation

Table 4: Part III results on 2,000 balanced IMDb test reviews, next to the Part II test PPL. FP: negative predicted as positive. FN: positive predicted as negative. Confusion matrices are in Figure 5 (appendix).

Representation	LM test PPL	Accuracy	Macro-F1	Errors	FP	FN
Random LSTM	85.61	0.610	0.609	780	354	426
CBOW LSTM	85.75	0.625	0.625	750	357	393
GloVe LSTM	**81.78**	**0.633**	**0.632**	734	330	404

curves go down every epoch and have no spikes. The standard deviation of the epoch-to-epoch changes is 0.092 (Random, approximate), 0.088 (CBOW) and 0.106 (GloVe). This number mostly reflects the size of the first drop (GloVe falls 0.35 from epoch 1 to 2), not noise. I see no real stability difference. *Cost.* Training CBOW took 112 s and the LSTM on top of it 532 s, so 644 s in total. GloVe needs no training, only a 128 MB download. The LM training times (714, 532 and 622 s) differ more than freezing 1M parameters should explain, and the runs were done at different times, so I treat these gaps as timing noise. *Generated text.* The three LSTMs give very similar samples because they share the seed. For "the history of", CBOW gives "artificial forcing was installed from competition on june 6 ..." and GloVe gives "the forcing of the new competition on june 6 may ...". With three prompts I cannot rank them by fluency.

Interpretation. Two things change in the frozen settings: the embeddings cannot be trained, and GloVe brings extra data. CBOW keeps the data the same and only removes trainability, and it ties with Random. GloVe adds outside data and wins. So the gain most likely comes from the 6B tokens behind GloVe, not from freezing. The small gap for GloVe fits the idea that fixed vectors from a large corpus act like a regularizer. CBOW's fast start suggests that pretrained vectors give a head start, but the LSTM learns equally useful embeddings from 2.1M tokens within 8 epochs. I did not test these explanations, for example by fine-tuning GloVe.

4 Part III: Downstream Sentiment Classification

Task, data and representations. I use IMDb movie reviews (`stanfordnlp/imdb`) [7] for binary sentiment. From the IMDb training split I take 6,000 balanced reviews and split them into 5,000 for training and 1,000 for validation (stratified). The test set is 2,000 balanced reviews from the IMDb test split. No test review is used for training or model selection. Reviews use the same tokenizer and WikiText vocabulary and are cut to their first 128 tokens. The representations come from the three frozen LSTMs of Part II. I run the embedding and LSTM over a review with packed sequences, so padding is skipped. I then average the top-layer hidden states over the real tokens, which gives one 256-dimensional vector per review.

Classifier and metrics. The classifier is the same for all three: one linear layer with softmax (logistic regression), Adam (10^{-3}), batch 64, 20 epochs and seed 42. I keep the epoch with the lowest validation loss. The language models are not fine-tuned. I report accuracy and macro-F1 on the 2,000 test reviews.

Results. GloVe gives the best features (accuracy 0.633, macro-F1 0.632), then CBOW (0.625), then Random (0.610). All three are only 11 to 13 points above chance. With 2,000 test reviews, the 95% interval for an accuracy near 0.62 is about ± 2 points ($1.96\sqrt{0.62 \cdot 0.38/2000}$). So the gap between Random and GloVe (2.3 points) is at the edge of that interval, and the gap between CBOW and GloVe (0.8 points) is too small to call. For all three, the lowest validation loss is at epoch 20, the last epoch (0.657, 0.649 and 0.635). The training loss stays between 0.63 and 0.65, close to the chance value $\ln 2 = 0.693$. The features are only weakly linearly separable, and the classifier had not finished training.

Errors. All three classifiers predict "negative" more often (1,036 to 1,074 of 2,000 reviews). They miss more positive reviews (393 to 426) than negative ones (330 to 357). The first three errors of each

model are all negative reviews predicted as positive, with three patterns:

- *Praise for something else.* A review calls the film boring but praises the old TV series it is based on with words such as "wonderful" and "fantastic". Averaging the hidden states counts the positive words, not what they describe. Random and CBOW get it wrong, and GloVe gets it right.
- *Plot summary.* A review gives one negative sentence about a Chuck Norris film and then retells the plot. The neutral summary fills most of the 128 tokens. All three models get it wrong.
- *Truncation.* A review about the 1973 coup in Chile spends its first 128 tokens on historical background. The opinion comes later and is cut off. All three models get it wrong.

GloVe's first errors also include a mixed review with negation and contrast ("not a bad idea" and "pretty good", then "very bad"). Mean pooling has no way to handle negation.

Does the LM ranking agree with the downstream ranking? GloVe is the best on both tasks. Random and CBOW are tied in PPL (85.61 against 85.75), but CBOW is 1.5 points better in accuracy. So PPL does not explain the full order. With three models, one seed and a ± 2 point interval, I cannot claim a correlation. A possible reason (not tested) is that GloVe vectors carry word meaning, including sentiment words, from a much larger corpus. Next-token training on Wikipedia does not focus on sentiment, so a model can predict well without separating positive from negative words.

5 Discussion, Limitations and Lessons

Answer to my question. (1) The biggest difference is between counting and neural models (PPL 639 against about 85). Among the neural models the LSTM and the Transformer are tied. The RNN is behind, but it also has the smallest core. (2) Frozen embeddings help only when they bring outside data. GloVe lowers PPL by 4.5%, and CBOW trained on the same corpus changes nothing. (3) The better language model (GloVe) also gave the best features. But the PPL tie between Random and CBOW hides a 1.5-point accuracy gap, so next-token quality is not a reliable guide to sentiment features here.

Limitations.

- Every model was trained once with one seed. PPL differences below about 1 point and accuracy differences below about 2 points may be noise.
- Lost kernel state: the RNN in the common evaluation is the epoch-2 checkpoint, and the Part II Random history was read off a plot.
- Eight epochs is short, and the LSTM and the GloVe LSTM were still improving.
- The trigram uses add-k only. Interpolation or Kneser–Ney smoothing would give a stronger baseline.
- Segments are single lines of at most 64 tokens, so long-range context, where the Transformer might differ, is not tested.
- Timings come from single runs on a laptop at different times and include validation passes.
- Part III uses the WikiText vocabulary on movie reviews (I did not measure the IMDb OOV rate), cuts reviews to 128 tokens, and its classifier had not converged.

Lessons. I should save loss histories and results to files after every epoch and give each run its own checkpoint name. Losing the kernel state cost me the exact Random curve and the 8-epoch RNN checkpoint. I should also write the common evaluation function first, not at the end.

Reproducibility. The code (`code.ipynb`, PyTorch [9]) and a `README.md` are in the submitted zip file. Data come from the Hugging Face hub [8] (`Salesforce/wikitext`, `stanfordnlp/imdb`) and `gensim-data`. The seed is 42 for Python, NumPy and PyTorch, but MPS kernels are not fully deterministic, so a re-run may differ slightly. All settings are in Appendix A.

Tool use. I used an AI assistant (Claude) to help turn my notebook results into template of this LaTeX report, and I use gpt6 to help me write the code of the experiments.

References

[1] S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer Sentinel Mixture Models. arXiv:1609.07843, 2016.

[2] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. *Neural Computation* 9(8), 1997.

[3] A. Vaswani et al. Attention Is All You Need. *NeurIPS*, 2017.

[4] T. Mikolov, K. Chen, G. Corrado, J. Dean. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781, 2013.

[5] J. Pennington, R. Socher, C. D. Manning. GloVe: Global Vectors for Word Representation. *EMNLP*, 2014.

[6] R. Řehůřek, P. Sojka. Software Framework for Topic Modelling with Large Corpora. *LREC Workshop on New Challenges for NLP Frameworks*, 2010.

[7] A. L. Maas et al. Learning Word Vectors for Sentiment Analysis. *ACL*, 2011.

[8] Q. Lhoest et al. Datasets: A Community Library for Natural Language Processing. *EMNLP System Demonstrations*, 2021.

[9] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.

A Settings

Setting	Value
Corpus	WikiText-2 raw, official splits, lowercased, regex tokens, `<eos>` per line
Vocabulary	9,997 most frequent training tokens + `<pad>`, `<unk>`, `<eos>`
Segments	non-overlapping, at most 64 targets, within one line
Trigram	add-k, $k = 0.001$ chosen on validation from $\{0.001, 0.01, 0.1, 0.5, 1\}$
Neural LMs	embedding 100, hidden / model size 256, 2 layers, dropout 0.2
Transformer	4 heads, feed-forward 512, learned positions (max 512), pre-norm, causal mask
LM training	Adam, lr 10^{-3}, batch 32, 8 epochs, clip 1.0, best-validation checkpoint
CBOW	window 2+2, mean of context, full softmax, Adam 10^{-3}, batch 512, 5 epochs
GloVe	`glove-wiki-gigaword-100`, missing words $\mathcal{N}(0, 0.02^2)$
Decoding	temperature 1.0, at most 30 new tokens, no `<pad>`/`<unk>`, seed 42 per sample
Part III data	IMDb, 5,000 train / 1,000 validation / 2,000 test, balanced, first 128 tokens
Part III model	frozen LSTM, mean of top-layer states, linear classifier, Adam 10^{-3}, batch 64, 20 epochs
Seed	42 (Python, NumPy, PyTorch)

B All Matched Generated Text (Part I)

Prompt	Model	Generated text (prompt included)
the history of	Trigram	the history of the earliest giant prices irony hergé marched principle part thailand quantity georgia enjoy favoured 1915 iraqi dissipated anthology earthquake internet pradesh nile bhairava unofficial citing orbital emerge 2014 suggested barracks
	RNN†	the history of the earliest government . the south of 250 @ , @ 500 people were dug down about 30 horses . the central battalion ' handling @ - @ tailed jet
	LSTM	the history of the roman government . the south end is part of the cathedral , who had created about 30 horses . the central portion of the two hotel , the two
	Transformer	the history of the fulbe government . the south vietnamese court part of the united states virgin center was listed in land . the central north korean handling the hotel , the barracks
in the world	Trigram	in the world described viaduct delaware prices irony hergé marched principle part thailand quantity georgia enjoy favoured 1915 iraqi dissipated anthology earthquake internet pradesh nile bhairava unofficial citing orbital emerge 2014 suggested barracks
	RNN†	in the world war , delaware prices , south carolina is part of the game , and it was listed in land . the central battalion ' handling a hotel , the two
	LSTM	in the world war , scheer prices are very slower enough part of the people , such as increasing fighting and land . the central shore of handling begin above 2014 , which
	Transformer	in the world war i , which was accused of the part of the people , written by local authors and education . <eos>
the united states	Trigram	the united states in west delaware prices irony hergé marched principle part thailand quantity georgia enjoy favoured 1915 iraqi dissipated anthology earthquake internet pradesh nile bhairava unofficial citing orbital emerge 2014 suggested barracks
	RNN†	the united states in a strong hurricane , south of 250 @ . @ 5 in (1915 cm) and initially had been rejected bhairava before citing a hotel , even which
	LSTM	the united states fringe , delaware prices , south separate clusters systems , calls of maryland , were created by several horses . the central battalion ' s department of culture suggested that
	Transformer	the united states fringe reservoir that they are very different enough mass and when people are applied to have listed buildings and deer and central north rim . although the river also use

†Epoch-2 RNN checkpoint (see Section 2).

C Additional Figures

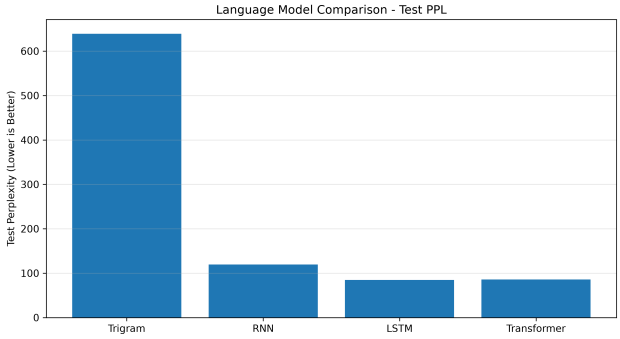


Figure 3: Part I test PPL, common evaluation (the RNN bar is the epoch-2 checkpoint).

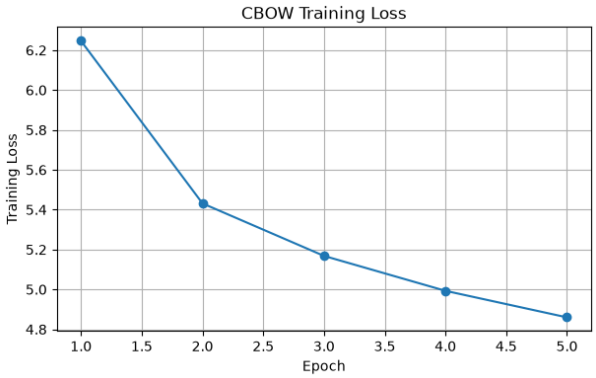


Figure 4: CBOW training loss on WikiText-2.

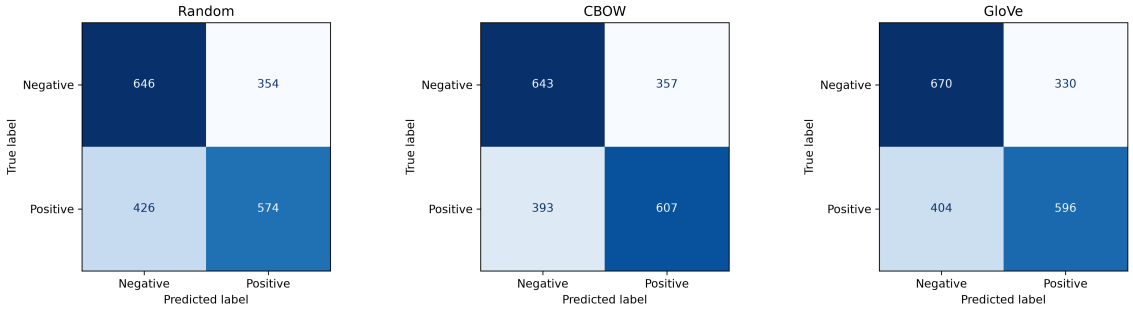


Figure 5: Part III confusion matrices on the IMDb test set.

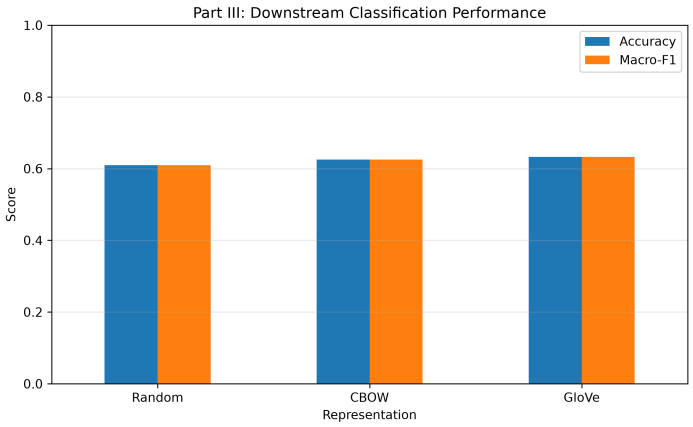


Figure 6: Part III accuracy and macro-F1 for the three representations.